Position Control with PID and Cascade Controllers in Simulink

Industrial Control course project. Prepared by Hossein Mosaffa.

1. Overview

This project is a Simulink model that compares three ways of closing a position loop around the same plant. The first loop has no compensator and simply feeds the output back to the reference. The second adds a single PID controller. The third is a cascade arrangement, with a PID controller in an outer position loop and a proportional gain in an inner loop. Each loop has its own pulse reference and its own scope, so the three responses can be run and viewed together.

The point of the project is to see how much a controller improves the response and what each structure costs. The model was saved in MATLAB R2016b and uses only standard Simulink blocks, which are transfer functions, sums, discrete pulse generators, scopes and the built-in PID Controller block.

2. The plant

All three loops control the same plant. It is a second-order transfer function

$$G(s) = 1.2 / (0.00077\, s^2 + 0.0539\, s + 1.441)$$

followed by an integrator 1/s. The second-order part has a DC gain of 0.833, a natural frequency of 43.3 rad/s and a damping ratio of 0.81. Its poles sit at $-35 \pm 25.4j$, so it is well damped and settles on its own within a fraction of a second. The integrator turns the output of that block into a position.

The model does not name the physical system. The structure is typical of a DC motor, where the second-order block describes the shaft speed as a function of the input and the integrator converts speed into angle, and this report follows that reading. Because the integrator sits inside every loop, each closed loop follows a step reference with no steady-state error, even without a controller.

3. Model structure

The model contains three independent loops drawn one above the other and labelled in the diagram as cascade controller, PID controller and without controller. The table lists the blocks of each loop in signal order.

Loop	Blocks in signal order	Feedback
Cascade controller	Pulse Generator1, Sum1, PID Controller1, Sum, Transfer Fcn3, Transfer Fcn, Transfer Fcn1, Scope	Position returns to Sum1 and the plant output returns to Sum
PID controller	Pulse Generator2, Sum2, PID Controller2, Transfer Fcn2, Transfer Fcn4, Scope1	Position returns to Sum2

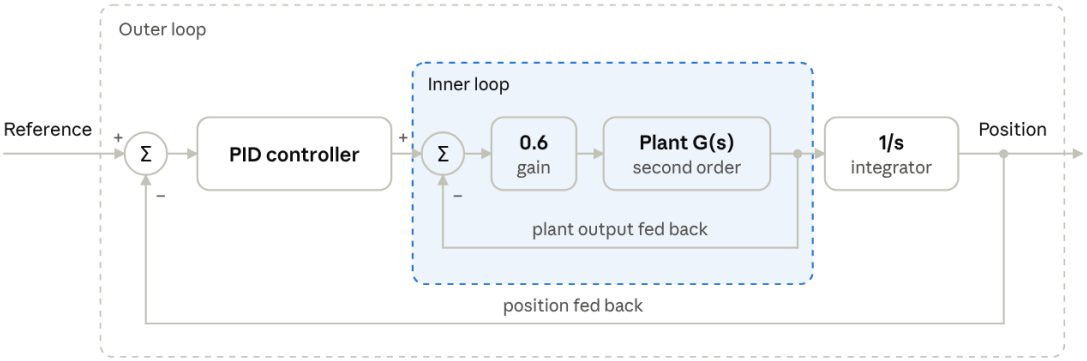
Loop	Blocks in signal order	Feedback
Without controller	Pulse Generator3, Sum3, Transfer Fcn5, Transfer Fcn6, Scope2	Position returns to Sum3

In each loop the transfer function with the second-order denominator is the plant, which is Transfer Fcn, Transfer Fcn2 or Transfer Fcn5, and the block with denominator s is the integrator. Every sum block adds the reference and subtracts the feedback. The loop without a controller is still a closed loop, since the position is fed back through a unity path, but nothing shapes the error signal before it reaches the plant.

A cascade controller uses two control loops nested one inside the other. The outer controller does not drive the plant directly. It watches the quantity we finally care about and sets the target for an inner controller, which watches a faster, closely related quantity and reacts to it. Think of a driver who wants to arrive on time. The driver picks a speed, which is the outer decision, and the cruise control holds that speed, which is the inner job. For this to work the inner loop has to respond faster than the outer one, so that it has settled before the outer loop changes its demand. The benefit is that the inner loop corrects disturbances and reshapes the plant before they reach the output, and each controller is usually easier to tune than a single one doing everything. The cost is an extra measurement and an extra controller to tune.

In the cascade loop the position error passes through PID Controller1. A second sum block then subtracts the output of the second-order block from the controller output. The difference is multiplied by Transfer Fcn3, which has a numerator of 0.6 and a denominator of 1 and so acts as a gain of 0.6, and then enters the plant. On the DC motor reading this is a proportional speed loop nested inside a PID position loop.

The inner loop closes around the plant, the outer around the position



cascade loop of the model, 2 feedback paths

The two feedback paths are what make this a cascade. The plant output closes the small inner loop and the position closes the large outer one.

4. Controller settings

Both PID blocks use the parallel form with a filtered derivative, so each controller is Kp + Ki/s + Kd·N·s/(s + N). With N = 100 the derivative filter has a time constant of 0.01 s. Neither block has an output limit or anti-windup, and all initial conditions are zero. The gains are in the table.

Parameter	PID Controller1 (cascade)	PID Controller2 (single loop)
Kp	63	50.4
Ki	6300/6 = 1050	50.4 × 14 = 705.6
Kd	0.04 × 63 = 2.52	50.4 / 56 = 0.9
N	100	100
Integral time Ti = Kp/Ki	0.06 s	0.071 s
Derivative time Td = Kd/Kp	0.04 s	0.018 s

The gains of PID Controller2 follow the closed-loop Ziegler–Nichols rules. For this plant with its integrator in a unity feedback loop, the system reaches sustained oscillation at an ultimate gain Ku of about 84.1, with an oscillation period Pu of about 0.145 s. The classic PID rules give Kp = 0.6 Ku, Ti = Pu/2 and Td = Pu/8. With Ku rounded to 84 and Pu rounded to 1/7 s, these give exactly the values in the model, which are 50.4 for Kp, 14 for 1/Ti and 56 for 1/Td. The model does not record how the gains were chosen, so this is an inference from the numbers, but the fit is very close.

The settings of PID Controller1 do not follow from the same rules, because its outer loop sees a different plant once the inner loop is closed. They appear to have been adjusted by hand. The inner proportional gain in that loop is 0.6.

5. Simulation setup

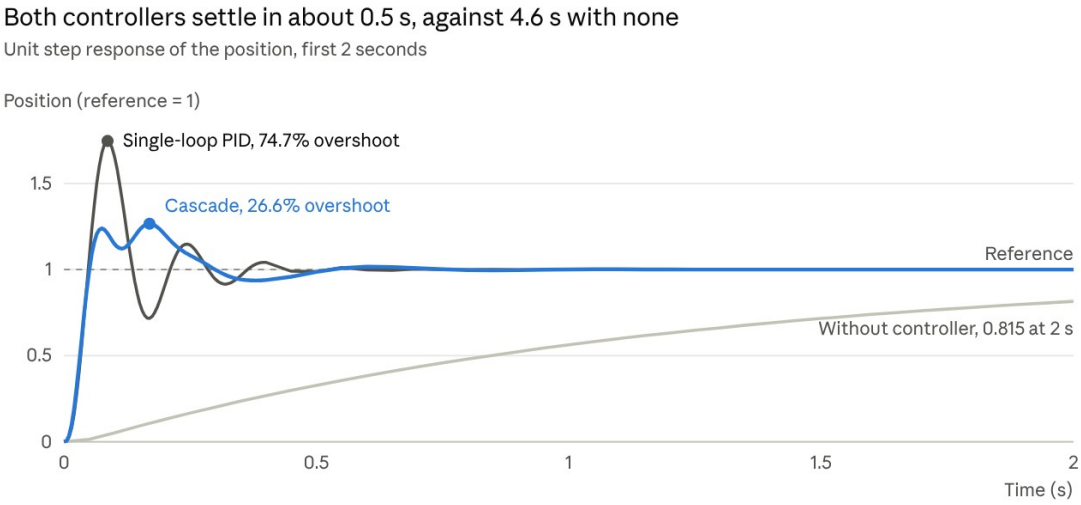
The model runs from 0 to 10 s with an automatically chosen variable-step solver and a relative tolerance of 0.001. Each loop is driven by a Discrete Pulse Generator in time based mode with amplitude 1 and a pulse width of 0.5. The other settings are the block defaults, which include a period of 2 and a sample time of 1. Each scope displays the position, which is the output of the last integrator in its loop.

To run the model, open PID.slx in MATLAB R2016b or newer, press Run and open the three scopes. No toolbox beyond Simulink should be needed, because every block comes from the standard Simulink libraries.

6. Results

The figures in this section were calculated outside Simulink from the same transfer functions and gains, using a unit step reference, which is the standard test signal. They were confirmed with two independent methods, one based on closed-loop transfer functions and one on a direct time simulation of the blocks. They are not read from the Simulink scopes, so the scope traces may differ slightly, particularly if the pulse reference has a different shape from a step.

Measure	Without controller	Single-loop PID	Cascade
Overshoot	none	74.7%	26.6%
Peak time	not applicable	0.086 s	0.169 s
Rise time, 10 to 90 percent	2.55 s	0.028 s	0.031 s
Settling time, 2 percent band	4.59 s	0.484 s	0.488 s
Steady-state error	0	0	0



Calculated from the model's transfer functions and gains with a unit step input, first 2 seconds

The plot covers the first two seconds, so the uncontrolled loop is still rising where it ends. It also shows that the single PID loop rings, dipping to about 0.72 and then overshooting again by about 15 percent, while the cascade loop dips only to about 0.94.

Without a controller the loop is stable and has no overshoot. Its slowest closed-loop pole is at about −0.86, a time constant of roughly 1.2 s, so it needs about 4.6 s to settle. This is the benchmark the controllers have to beat.

Both controlled loops are roughly nine to ten times faster, with 2 percent settling times near 0.49 s and rise times near 0.03 s. The difference lies in the overshoot. The single PID loop overshoots by almost 75 percent, which is a lot, while the cascade loop overshoots by about 27 percent, roughly a third as much. The two PID blocks have different gains, so this is not a comparison of structure alone. Still, the inner loop feeds back the output of the second-order block, which on the motor reading is speed, and speed feedback adds damping. That fits the smaller overshoot.

The controller output is also large at the start. Because the derivative acts on the error and the reference steps, the derivative term jumps at once to Kd·N, which is 90 in the single loop. Together with the proportional term the signal entering the plant is about 140 at the first instant. In the cascade loop it is about 189.

7. Limitations and possible improvements

The model is an ideal linear system. It has no actuator limit, no load disturbance, no sensor noise and no friction or backlash. The unlimited controller output matters most. The jumps of 140 and 189 at the start of a step would saturate a real drive, and with saturation the integral term would wind up, since neither PID block has anti-windup enabled. Adding output limits with anti-windup would be the first step toward a realistic model.

The overshoot of the single PID loop is the other weak point. Ziegler–Nichols settings are known to be aggressive, and 75 percent overshoot is well above what most applications tolerate. Lowering Kp, or using the two-degree-of-freedom form of the PID block with set-point weighting on the proportional term, would reduce it at the cost of some speed. Taking the derivative from the measurement instead of the error would remove the large initial spike in the controller output. A fairer comparison of the two structures would also retune both controllers with the same method and the same objective.

8. Arduino implementation

The same idea was also implemented on hardware with an Arduino. The sketch reads a quadrature encoder on the motor shaft through two interrupt pins, converts the count to an angle and runs a discrete PID controller every 100 ms. The result drives one PWM pin and two direction pins on the motor driver. The setpoint is 100 degrees, and the gains in the original sketch are Kp 0.32, Ki 0.1 and Kd 0.3, with a comment saying they are meant to be adjusted by trial and error. These gains cannot be compared with the Simulink ones, because the error here is in degrees, the output is a PWM value from 0 to 255 and the loop runs at about 10 Hz.

The original sketch had one real bug and several weaknesses, and the version below corrects them. The bug was that the motor direction came from the sign of the error while the drive strength came from the absolute value of the PID output. When the derivative term pushed the output negative near the target, the absolute value turned that braking action into a push toward the target. The corrected code takes the direction from the sign of the PID output itself. The other changes are smaller. The integral term is now clamped so that it cannot wind up beyond the output range. The shared encoder count is declared volatile and read with interrupts briefly disabled. The two identical interrupt routines are merged into one. The loop waits with the millis timer instead of a blocking delay, and the interrupts are attached with digitalPinToInterrupt. The decoding logic, the pins and the gains are unchanged.

```
// PID position control of a DC motor with a quadrature encoder
const int PWM_PIN = 11;   // PWM output to the motor driver
const int DIR1 = 4;       // direction pins
const int DIR2 = 5;
const int ENC_A = 2;      // encoder channels, both on interrupt pins
const int ENC_B = 3;

const double DEG_PER_COUNT = 0.9;     // set from the encoder, see the notes below
const double SETPOINT = 100.0;        // target angle in degrees
const double Kp = 0.32, Ki = 0.1, Kd = 0.3;
const unsigned long SAMPLE_MS = 100;  // control period
```

```
volatile long count = 0;   // shared with the interrupt routine
volatile byte statep = 0;  // previous encoder state

double totalError = 0;
double lastError = 0;
unsigned long lastSample = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ENC_A, INPUT);
  pinMode(ENC_B, INPUT);
  pinMode(PWM_PIN, OUTPUT);
  pinMode(DIR1, OUTPUT);
  pinMode(DIR2, OUTPUT);
  attachInterrupt(digitalPinToInterrupt(ENC_A), encoderChange, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ENC_B), encoderChange, CHANGE);
}

void loop() {
  unsigned long now = millis();
  if (now - lastSample < SAMPLE_MS) return;   // run once per control period
  lastSample = now;

  noInterrupts();
  long c = count;         // read the shared counter without interruption
  interrupts();
  double angle = DEG_PER_COUNT * c;

  double error = SETPOINT - angle;
  double changeError = error - lastError;     // derivative term
  totalError += error;                        // integral term
  totalError = constrain(totalError, -255.0 / Ki, 255.0 / Ki);   // anti-windup

  double pidTerm = Kp * error + Ki * totalError + Kd * changeError;
  pidTerm = constrain(pidTerm, -255.0, 255.0);
  lastError = error;

  // the direction follows the sign of the PID output, not the sign of the error
  if (pidTerm > 0) {
    digitalWrite(DIR1, LOW);    // forward
    digitalWrite(DIR2, HIGH);
  } else {
    digitalWrite(DIR1, HIGH);   // reverse
    digitalWrite(DIR2, LOW);
  }
  analogWrite(PWM_PIN, (int)fabs(pidTerm));
```

```
  Serial.print("Angle: ");
  Serial.println(angle);
}

// one routine for both encoder channels, same decoding as the original
void encoderChange() {
  bool a = digitalRead(ENC_A);
  bool b = digitalRead(ENC_B);
  byte state;
  if (a && b) state = 1;
  else if (a && !b) state = 2;
  else if (!a && !b) state = 3;
  else state = 4;

  switch (state) {
    case 1:
      if (statep == 2) count++;
      if (statep == 4) count--;
      break;
    case 2:
      if (statep == 1) count--;
      if (statep == 3) count++;
      break;
    case 3:
      if (statep == 2) count--;
      if (statep == 4) count++;
      break;
    default:
      if (statep == 1) count++;
      if (statep == 3) count--;
  }
  statep = state;
}
```

This code has not been run on hardware, so test it with the motor unloaded and at a reduced supply voltage first. The one value that must be set for your encoder is DEG_PER_COUNT. The original PID sketch used 0.9 degrees per count and the separate encoder test sketch used 0.09, and these cannot both be right. As far as the code shows, the decoder counts every edge of both channels, so the value should be 360 divided by four times the encoder's pulses per revolution. On that basis 0.9 fits a 100 pulse encoder and 0.09 fits a 1000 pulse encoder. A quick check is to turn the shaft by hand through one revolution and confirm that the count printed by the test sketch equals 360 divided by DEG_PER_COUNT.

9. Conclusion

Closing the position loop around this plant gives a stable response with no steady-state error, but a slow one, with a settling time of about 4.6 s. A PID controller tuned by the Ziegler–Nichols rules cuts that to

about 0.5 s at the price of heavy overshoot and a large initial control signal. The cascade arrangement settles just as quickly and overshoots about a third as much, at the cost of a second loop and a measurement of the inner variable. Making the model more realistic would start with actuator limits, anti-windup and a disturbance input.